ADC Stack Architecture of ESP32

Detailed Documentation using ESP-IDF Driver / HAL Model

1. Introduction

An Analog-to-Digital Converter (ADC) converts an analog voltage signal into a digital value that can be processed by a microcontroller.

Many real-world sensors produce analog signals, such as:

- Temperature sensors (LM35)

- Light sensors

- Gas sensors

- Potentiometers

- Battery monitoring circuits

- Biomedical sensors

The ADC peripheral of the ESP32 allows these analog signals to be digitized and processed by software.

2. ADC Architecture in ESP32

The ESP32 ADC uses a Successive Approximation Register (SAR) architecture.

The conversion process can be visualized as:

Analog Input Voltage

↓

Sample and Hold Circuit

↓

SAR Comparator

↓

Digital Output Register

ADC Resolution : 12-bit resolution

This means the ADC can represent:0 – 4095

Example mapping:

Voltage	Digital Value
0v	0
1.65v	2048 approx.:
3.3v	4095 approx

3. ADC Units Available in ESP32

The ESP32 contains two ADC peripherals.

ADC Unit	Channels	Notes
ADC1	8 Channels	Works with wifi
ADC2	10 Channels	Shared with wifi

Important constraint: ADC2 cannot be used reliably when WiFi is active. Therefore ADC1 is recommended for most applications.

4. ADC Channel Mapping (ADC1)

ADC Channel | GPIO Pin

ADC Channel	GPIO Pin
CH0	36
CH1	37
CH2	38
CH3	39
CH4	32
CH5	33
CH7	34
CH8	35

Example sensor connection:

LM35 → GPIO34

Which corresponds to:

ADC1_CHANNEL_6

5. ESP32 ADC Software Stack

ESP-IDF uses a layered driver architecture.

Application Layer

↓

ESP-IDF ADC Driver

↓

ADC HAL (Hardware Abstraction Layer)

↓

Low Level Register Interface

↓

ADC Hardware Peripheral

Application Layer

User program that reads sensor data.

Example API call:

adc1_get_raw()

Driver Layer

Provides high-level APIs for:

- configuring ADC

- reading ADC values

- calibration support

HAL Layer

Translates driver functions into hardware register operations.

Register Layer

Contains memory-mapped registers controlling the ADC hardware.

Hardware Layer

Actual SAR ADC peripheral performing the conversion.

6. Header Files Used for ADC Programming

Two main header files are used in ESP-IDF ADC programming.

```
#include "driver/adc.h"

#include "esp_adc_cal.h"
```

These headers provide configuration, reading, and calibration functions.

7. Functions in driver/adc.h

This header contains core ADC driver functions.

These functions allow the user to:

- configure ADC resolution

- configure input attenuation

- read raw ADC values

7.1 Configure ADC Resolution

Function:

adc1_config_width()

Prototype:

```
esp_err_t adc1_config_width(adc_bits_width_t width_bit);
```

Example usage:

```
adc1_config_width(ADC_WIDTH_BIT_12);
```

Available resolutions:

Mode	Resolution
ADC_WIDTH_BIT_9	9-bit
ADC_WIDTH_BIT_10	10-bit
ADC_WIDTH_BIT_11	11-bit
ADC_WIDTH_BIT_12	12-bit

7.2 Configure Channel Attenuation

Function:

```
adc1_config_channel_atten()
```

Prototype:

```
esp_err_t adc1_config_channel_atten(
    adc1_channel_t channel,
    adc_atten_t atten);
```

Example usage:

```
adc1_config_channel_atten(ADC1_CHANNEL_6, ADC_ATTEN_DB_11);
```

Attenuation determines the maximum measurable voltage range.

Attenuation	Voltage Range
0dB	0-1.1V
2.5dB	0-1.5V
6dB	0-2.2V
11dB	0-3.3V

Most sensor applications use:

ADC_ATTEN_DB_11

7.3 Read Raw ADC Value

Function:

adc1_get_raw()

Prototype:

```
int adc1_get_raw(adc1_channel_t channel);
```

Example usage:

```
int raw = adc1_get_raw(ADC1_CHANNEL_6);
```

Output range: 0 – 4095

7.4 ADC2 Driver Functions

For ADC2 channels, the following functions exist.

Configure ADC2 Channel

adc2_config_channel_atten()

Prototype:

```
esp_err_t adc2_config_channel_atten(
    adc2_channel_t channel,
```

```
    adc_atten_t atten);
```

Read ADC2 Value

adc2_get_raw()

Prototype:

```
esp_err_t adc2_get_raw(
    adc2_channel_t channel,
    adc_bits_width_t width_bit,
    int *raw_out);
```

Note:

ADC2 may conflict with WiFi

8. Functions in esp_adc_cal.h

The ESP32 ADC suffers from:

- reference voltage variation

- non-linear response

- chip-to-chip differences

Therefore calibration functions are provided.

8.1 Calibration Structure

Calibration data is stored in the following structure:

esp_adc_cal_characteristics_t adc_chars;

This structure contains:

- ADC reference voltage

- calibration coefficients

- attenuation information

8.2 ADC Characterization Function

Function:

esp_adc_cal_characterize()

Prototype:

```
esp_adc_cal_value_t esp_adc_cal_characterize(
    adc_unit_t adc_num,
    adc_atten_t atten,
    adc_bits_width_t bit_width,
    uint32_t default_vref,
    esp_adc_cal_characteristics_t *chars);
```

Example usage:

```
esp_adc_cal_characterize(
    ADC_UNIT_1,
    ADC_ATTEN_DB_11,
    ADC_WIDTH_BIT_12,
    1100,
    &adc_chars);
```

Meaning of Parameters

ADC_UNIT_1 |	ADC module used
ADC_ATTEN_DB_11	attenuation level
ADC_WIDTH_BIT_12	resolution

1100	1100 | reference voltage estimate (mV)
&adc_chars	address of calibration structure

8.3 Meaning of &adc_chars

adc_chars is a structure variable.

esp_adc_cal_characteristics_t adc_chars;

The symbol:

&adc_chars

means:

address of adc_chars

The function fills this structure with calibration data.

8.4 Convert Raw ADC Value to Voltage

Function:

esp_adc_cal_raw_to_voltage()

Prototype:

```
uint32_t esp_adc_cal_raw_to_voltage(
    uint32_t adc_reading,
    const esp_adc_cal_characteristics_t *chars);
```

Example usage:

```
uint32_t voltage = esp_adc_cal_raw_to_voltage(raw, &adc_chars);
```

Output:

Voltage in millivolts

9. Complete Signal Flow

Sensor (Analog Output)

↓

ESP32 ADC Hardware

↓

ADC Register

↓

HAL Layer

↓

ADC Driver

↓

Application Code

↓

printf()

↓

UART0

↓

Serial Monitor

10. Best Practices

- Use ADC1

  ADC1 works with WiFi

ADC2 conflicts with WiFi

- Use Calibration
- Convert raw values using:

esp_adc_cal_raw_to_voltage()

- Average Multiple Samples
- Better accuracy can be obtained by averaging readings.

Example approach:

Take 10–20 samples

Compute average

Use the averaged value

11. Summary

The ESP32 ADC stack uses a layered architecture that simplifies sensor interfacing.

Application

↓

ESP-IDF ADC Driver

↓

HAL Layer

↓

Register Layer

↓

ADC Hardware

Advantages:

- simplified programming

- hardware abstraction

- calibration support

- improved measurement accuracy